CS3501 – Compiler Design

20 Lab Programs with Algorithms and Sample Outputs

Each experiment is arranged as Program → Algorithm → Sample Output.

1. Lexical Analyzer to Recognize Identifiers and Constants

PROGRAM

```
#include &lt;stdio.h&gt;
 #include &lt;ctype.h&gt;
 #include &lt;string.h&gt;

 int main() {
     char s[100];
     int i;

     printf("Enter a string: ");
     scanf("%s", s);

     if (isalpha(s[0]) || s[0] == '_') {
         for (i = 1; s[i]; i++)
             if (!isalnum(s[i]) &amp;&amp; s[i] != '_')
                 break;

          if (s[i] == '\0')
               printf("Identifier");
          else
               printf("Invalid");
     }
     else if (isdigit(s[0])) {
         for (i = 1; s[i]; i++)
             if (!isdigit(s[i]))
                 break;

          if (s[i] == '\0')
               printf("Constant");
          else
               printf("Invalid");
     }
     else
          printf("Invalid");

     return 0;
 }
```

ALGORITHM

1. Start the program.
2. Read the input string.
3. Check whether the first character is a letter or underscore; if so, verify remaining characters are letters, digits or underscores.
4. Otherwise, check whether all characters are digits.
5. Print Identifier, Constant or Invalid.
6. Stop.

SAMPLE OUTPUT

```
Example input: total\nOutput: Identifier\nExample input: 12345\nOutput: Constant
```

2. Symbol Table – Insert, Delete, Search and Modify

PROGRAM

```
#include &lt;stdio.h&gt;
 #include &lt;string.h&gt;

 struct symbol {
     char name[20];
     char value[20];
 } s[50];

 int n = 0, i;
 char name[20], value[20];

 int main() {
     int ch;

     while (1) {
         printf("\\n1.Insert 2.Delete 3.Search 4.Modify 5.Display 6.Exit\\n");
         scanf("%d", &amp;ch);

          switch (ch) {
```

```
            case 1:
                scanf("%s%s", s[n].name, s[n].value);
                n++;
                break;

            case 2:
                scanf("%s", name);
                for (i = 0; i &lt; n; i++)
                    if (!strcmp(s[i].name, name)) {
                        s[i] = s[n - 1];
                        n--;
                        break;

                    }
                break;

            case 3:
                scanf("%s", name);
                for (i = 0; i &lt; n; i++)
                    if (!strcmp(s[i].name, name)) break;
                printf("%s", i &lt; n ? s[i].value : "Not found");
                break;

            case 4:
                scanf("%s%s", name, value);
                for (i = 0; i &lt; n; i++)
                    if (!strcmp(s[i].name, name))
                        strcpy(s[i].value, value);
                break;

            case 5:
                for (i = 0; i &lt; n; i++)
                    printf("%s %s\\n", s[i].name, s[i].value);
                break;

            case 6:
                return 0;
            }
        }
    }
```

ALGORITHM

1. Initialize an empty symbol table.

2. Display the operation menu.

3. For Insert, read and store name and value.

4. For Delete, search the name and remove its entry.

5. For Search, locate and display the value.

6. For Modify, locate the name and update its value.

7. Repeat until Exit is selected.

SAMPLE OUTPUT

```
Example session: Insert x 10; Search x\nOutput: 10\nModify x 20; Search x\nOutput: 20
```

3. Lex Program to Separate Tokens

PROGRAM

```
%{
 #include &lt;stdio.h&gt;
 %}
 %%
 "int"|"float"|"char"|"if"|"else"|"while" { printf("Keyword: %s\\n", yytext); }
 [a-zA-Z_][a-zA-Z0-9_]*                   { printf("Identifier: %s\\n", yytext); }
 [0-9]+                                   { printf("Constant: %s\\n", yytext); }
 [+\\-*/=&lt;&gt;]                              { printf("Operator: %s\\n", yytext); }
 [ \\t\\n]+                               ;
 .                                        { printf("Special: %s\\n", yytext); }
 %%
 int main() {
     yylex();
     return 0;
 }
 int yywrap() { return 1; }
```

ALGORITHM

1. Start the Lex scanner.

2. Read the source text.

3. Match each lexeme against keyword, identifier, number and operator patterns.

4. Print the token category and lexeme.

5. Ignore whitespace and handle other special characters.

6. Stop at end of input.

SAMPLE OUTPUT

```
Input: int a = 25;\nOutput: Keyword: int; Identifier: a; Operator: =; Constant: 25
```

4. YACC Program to Validate Arithmetic Expressions

PROGRAM

```
/* expr.l */
 %{
 #include "y.tab.h"
 %}
 %%
 [0-9]+       { return NUM; }
 [ \\t\\n]     ;
 .            { return yytext[0]; }
 %%
 int yywrap() { return 1; }

 /* expr.y */
 %{
 #include &lt;stdio.h&gt;
 %}
 %token NUM
 %left '+' '-'
 %left '*' '/'
 %%
 E : E '+' E
    | E '-' E
    | E '*' E
    | E '/' E
    | '(' E ')'
    | NUM
    ;
 %%
 int main() {
      printf("Enter expression: ");
      if (yyparse() == 0)
          printf("Valid Expression\\n");

     return 0;
 }
 int yyerror() {
     printf("Invalid Expression\\n");
     return 0;
 }
```

ALGORITHM

1. The Lex program returns NUM for numbers and character tokens for operators and parentheses.

2. The YACC parser reads the tokens.

3. Apply the grammar rules for operands, operators and parenthesized expressions.

4. If the input satisfies the grammar, report Valid Expression.

5. Otherwise, report Invalid Expression.

SAMPLE OUTPUT

```
Input: (12+3)*4\nOutput: Valid Expression
```

5. YACC Program to Recognize a Valid Variable

PROGRAM

```
/* var.l */
 %{
 #include "y.tab.h"
 %}
 %%
 [a-zA-Z]       { return ALPHA; }
 [0-9]          { return DIGIT; }
 \\n             { return '\\n'; }
 .              { return INVALID; }
 %%
 int yywrap() { return 1; }

 /* var.y */
 %{
 #include &lt;stdio.h&gt;
 %}
 %token ALPHA DIGIT INVALID
 %%
 S : ALPHA T '\\n' { printf("Valid Variable\\n"); }
    ;
```

```
T : ALPHA T
   | DIGIT T
   |
   ;
%%
int main() {
     printf("Enter variable: ");
     yyparse();
     return 0;
}
int yyerror() {
     printf("Invalid Variable\\n");
     return 0;
}
```

ALGORITHM

1. The Lex scanner returns ALPHA for letters and DIGIT for digits.
2. The YACC parser expects the first token to be ALPHA.
3. Accept any number of following ALPHA or DIGIT tokens.
4. If the complete input matches, report Valid Variable.
5. Otherwise, report Invalid Variable.

SAMPLE OUTPUT

```
Input: abc123\nOutput: Valid Variable
```

6. Calculator Using LEX and YACC

PROGRAM

```
/* calc.l */
 %{
 #include "y.tab.h"
 #include &lt;stdlib.h&gt;
 %}
 %%
 [0-9]+       { yylval = atoi(yytext); return NUM; }
 [ \\t\\n]     ;
 .            { return yytext[0]; }
 %%
 int yywrap() { return 1; }

 /* calc.y */
 %{
 #include &lt;stdio.h&gt;
 %}
 %token NUM
 %left '+' '-'
 %left '*' '/'
 %%
 E : E '+' E { $$ = $1 + $3; }
    | E '-' E { $$ = $1 - $3; }
    | E '*' E { $$ = $1 * $3; }
    | E '/' E { $$ = $1 / $3; }
    | '(' E ')' { $$ = $2; }
    | NUM { $$ = $1; }
    ;
 %%
 int main() {
     printf("Enter expression: ");
     yyparse();
     return 0;
 }
 int yyerror() {

     printf("Invalid Expression\\n");
     return 0;
 }
```

ALGORITHM

1. The Lex scanner converts number strings into numeric tokens.
2. The YACC parser reads the arithmetic expression.
3. Evaluate addition, subtraction, multiplication and division using grammar actions.
4. Respect operator precedence and parentheses.
5. Print the calculated result or report invalid syntax.

SAMPLE OUTPUT

```
Input: 8+2*3\nOutput: The grammar evaluates the expression to 14 (add a print action to display the
value).
```

7. Generate Three-Address Code

PROGRAM

```
#include &lt;stdio.h&gt;

int main() {
    char a, b, op;
    int n = 0;

    printf("Enter expression (a+b): ");
    scanf(" %c %c %c", &amp;a, &amp;op, &amp;b);

    printf("t%d = %c %c %c\\n", ++n, a, op, b);
    return 0;
}
```

ALGORITHM

1. Read the left operand, operator and right operand.
2. Create a temporary variable.
3. Generate one three-address instruction using the operands and operator.
4. Display the generated instruction.
5. Stop.

SAMPLE OUTPUT

```
Input: a + b\nOutput: t1 = a + b
```

8. Simple Type Checker

PROGRAM

```
#include &lt;stdio.h&gt;

int main() {
    char type1[10], type2[10];

    printf("Enter type of first variable: ");
    scanf("%s", type1);

    printf("Enter type of second variable: ");
    scanf("%s", type2);

    if (type1[0] == type2[0])
        printf("Type Compatible");
    else
        printf("Type Error");

    return 0;
}
```

ALGORITHM

1. Read the data types of two variables.
2. Compare the entered types.
3. If the types match, report Type Compatible.
4. Otherwise, report Type Error.
5. Stop.

SAMPLE OUTPUT

```
Input types: int and int\nOutput: Type Compatible
```

9. Code Optimization – Constant Folding

PROGRAM

```
#include &lt;stdio.h&gt;

int main() {
    int a = 10, b = 20, c;

    c = a + b;

    printf("Before optimization: c = a + b\\n");
    printf("After optimization: c = 30\\n");
    printf("Result = %d", c);

    return 0;
}
```

ALGORITHM

1. Assign constant values to variables.
2. Evaluate the expression at compile time.
3. Replace the expression with its calculated constant value.
4. Display the expression before and after optimization.
5. Stop.

SAMPLE OUTPUT

```
Before: c = a + b\nAfter: c = 30
```

10. Generate Machine Code from Intermediate Code

PROGRAM

```
#include &lt;stdio.h&gt;

 int main() {
     char a, b, op;

     printf("Enter TAC (a+b): ");
     scanf(" %c %c %c", &amp;a, &amp;op, &amp;b);

     printf("MOV R0, %c\\n", a);

     switch (op) {
         case '+': printf("ADD R0, %c\\n", b); break;
         case '-': printf("SUB R0, %c\\n", b); break;
         case '*': printf("MUL R0, %c\\n", b); break;
         case '/': printf("DIV R0, %c\\n", b); break;
     }

     printf("MOV c, R0\\n");
     return 0;

 }
```

ALGORITHM

1. Read a simple three-address expression.
2. Load the first operand into a register.
3. Select the machine instruction corresponding to the operator.
4. Apply the instruction to the second operand.
5. Store the result in the destination variable.
6. Stop.

SAMPLE OUTPUT

```
Input: a + b\nOutput: MOV R0, a; ADD R0, b; MOV c, R0
```

11. YACC Pattern: Uppercase Letter Followed by Five Alphanumeric Characters

PROGRAM

```
/* pattern.l */
 %{
 #include "y.tab.h"
 %}
 %%
 [A-Z][a-zA-Z0-9]{5} { return VALID; }
 \\n                  { return '\\n'; }
 .                   { return INVALID; }
 %%
 int yywrap() { return 1; }

 /* pattern.y */
 %{
 #include &lt;stdio.h&gt;
 %}
 %token VALID INVALID
 %%
 S : VALID '\\n' { printf("Valid Pattern\\n"); }
   ;
 %%
 int main() {
      printf("Enter pattern: ");
      yyparse();
      return 0;
 }
 int yyerror() {
```

```
    printf("Invalid Pattern\\n");
    return 0;
}
```

ALGORITHM

1. The Lex scanner recognizes a capital letter followed by five alphanumeric characters.
2. Return VALID when the complete pattern matches.
3. The YACC parser accepts the VALID token.
4. Print Valid Pattern; otherwise report Invalid Pattern.
5. Stop.

SAMPLE OUTPUT

```
Input: A12bc3\nOutput: Valid Pattern
```

12. Lexical Analyzer for Comments, Operators and Keywords

PROGRAM

```
%{
 #include &lt;stdio.h&gt;
 %}
 %%
 "//".*                          { printf("Comment\\n"); }
 "/*"([^*]|\\*+[^*/])*\\*+"/"    { printf("Comment\\n"); }
 "int"|"float"|"char"|"return" { printf("Keyword: %s\\n", yytext); }
 [+\\-*/=&lt;&gt;]+                    { printf("Operator: %s\\n", yytext); }
 [a-zA-Z_][a-zA-Z0-9_]*          { printf("Identifier: %s\\n", yytext); }
 [0-9]+                          { printf("Constant: %s\\n", yytext); }
 [ \\t\\n]+                      ;
 .                               ;
 %%
 int main() { yylex(); return 0; }
 int yywrap() { return 1; }
```

ALGORITHM

1. Read the input source text.
2. Recognize single-line and block comments.
3. Recognize reserved keywords, operators, identifiers and constants using Lex rules.
4. Print the corresponding token category.
5. Ignore whitespace and unrelated characters.
6. Stop.

SAMPLE OUTPUT

```
Input: // comment\nint a = 10;\nOutput: Comment; Keyword: int; Identifier: a; Operator: =; Constant:
 10
```

13. Recognize C Control Structures Using YACC

PROGRAM

```
/* control.l */
 %{
 #include "y.tab.h"
 %}
 %%
 "if"       { return IF; }
 "else"     { return ELSE; }
 "while"    { return WHILE; }
 "for"      { return FOR; }
 [a-zA-Z]+ { return ID; }
 [0-9]+     { return NUM; }
 [ \\t\\n]   ;
 .          { return yytext[0]; }
 %%
 int yywrap() { return 1; }

 /* control.y */
 %{
 #include &lt;stdio.h&gt;
 %}
 %token IF ELSE WHILE FOR ID NUM
 %%
 S : IF '(' E ')' S
    | IF '(' E ')' S ELSE S
    | WHILE '(' E ')' S

    | FOR '(' E ';' E ';' E ')' S
```

```
    | '{' S '}'
    | ID ';'
    | ';'
    ;
E : ID | NUM | E '&lt;' E | E '&gt;' E
    ;
%%
int main() {
     printf("Enter control structure: ");
     yyparse();
     return 0;
}
int yyerror() {
     printf("Invalid Syntax\\n");
     return 0;
}
```

ALGORITHM

1. The Lex scanner returns tokens for if, else, while, for, identifiers and numbers.
2. The YACC parser checks the tokens against control-structure grammar rules.
3. Accept valid statements and blocks.
4. Report Invalid Syntax when parsing fails.
5. Stop.

SAMPLE OUTPUT

```
Input: if (x < 10) ;\nOutput: Accepted by the grammar
```

14. Count Vowels and Consonants Using LEX

PROGRAM

```
%{
 #include &lt;stdio.h&gt;
 int v = 0, c = 0;
 %}
 %%
 [aeiouAEIOU] { v++; }
 [a-zA-Z]      { c++; }
 \\n             { printf("Vowels=%d Consonants=%d\\n", v, c); return 0; }
 .             ;
 %%
 int main() {
     printf("Enter string: ");
     yylex();
     return 0;
 }
 int yywrap() { return 1; }
```

ALGORITHM

1. Initialize vowel and consonant counters to zero.
2. Read each character using Lex.
3. Increment the vowel counter for vowels.
4. Increment the consonant counter for other alphabetic characters.
5. At newline, display both counts.
6. Stop.

SAMPLE OUTPUT

```
Input: Education\nOutput: Vowels=5 Consonants=4
```

15. Generate Machine Code from Postfix Notation

PROGRAM

```
#include &lt;stdio.h&gt;
 #include &lt;ctype.h&gt;

 int main() {
     char s[100];
     int i;

     printf("Enter postfix expression: ");
     scanf("%s", s);

     for (i = 0; s[i]; i++) {
         if (isalnum((unsigned char)s[i]))
             printf("PUSH %c\\n", s[i]);
         else {
```

```
            printf("POP R1\\nPOP R0\\n");
            if (s[i] == '+') printf("ADD R0, R1\\n");
            if (s[i] == '-') printf("SUB R0, R1\\n");
            if (s[i] == '*') printf("MUL R0, R1\\n");
            if (s[i] == '/') printf("DIV R0, R1\\n");
            printf("PUSH R0\\n");
        }
    }
    return 0;
}
```

ALGORITHM

1. Read the postfix expression from left to right.
2. For each operand, generate a PUSH instruction.
3. For each operator, generate POP instructions for its operands.
4. Generate the corresponding arithmetic instruction.
5. Push the intermediate result back onto the stack.
6. Stop after processing the expression.

SAMPLE OUTPUT

```
Input: ab+c*\nOutput: PUSH a; PUSH b; ADD; PUSH c; MUL
```

16. LEX Program to Scan C Reserved Words, Variables and Operators

PROGRAM

```
%{
 #include &lt;stdio.h&gt;
 %}
 %%
 "int"|"char"|"float"|"if"|"else"|"return" {
     printf("Keyword: %s\\n", yytext);
 }
 [a-zA-Z_][a-zA-Z0-9_]* {
     printf("Variable: %s\\n", yytext);
 }
 [+\\-*/=&lt;&gt;] {
     printf("Operator: %s\\n", yytext);
 }

 [0-9]+ {
     printf("Number: %s\\n", yytext);
 }
 [ \\t\\n]+ ;
 . ;
 %%
 int main() { yylex(); return 0; }
 int yywrap() { return 1; }
```

ALGORITHM

1. Read the source text.
2. Match reserved words first.
3. Recognize identifiers, operators and numeric constants using Lex patterns.
4. Print each token and its category.
5. Ignore whitespace and other characters.
6. Stop.

SAMPLE OUTPUT

```
Input: int value = 10;\nOutput: Keyword: int; Variable: value; Operator: =; Number: 10
```

17. Convert Three-Address Code into Assembly Language

PROGRAM

```
#include &lt;stdio.h&gt;

 int main() {
     char result, a, b, op;

     printf("Enter TAC (a=b+c): ");
     scanf(" %c=%c%c%c", &amp;result, &amp;a, &amp;op, &amp;b);

     printf("MOV AX, %c\\n", a);

     switch (op) {
         case '+': printf("ADD AX, %c\\n", b); break;
```

```
        case '-': printf("SUB AX, %c\\n", b); break;
        case '*': printf("MUL %c\\n", b); break;
        case '/': printf("DIV %c\\n", b); break;
    }

    printf("MOV %c, AX\\n", result);
    return 0;
}
```

ALGORITHM

1. Read a three-address assignment of the form result = operand operator operand.
2. Move the first operand into a register.
3. Generate the assembly instruction for the operator.
4. Store the register result in the destination variable.
5. Stop.

SAMPLE OUTPUT

```
Input: a=b+c\nOutput: MOV AX, b; ADD AX, c; MOV a, AX
```

18. Eliminate Left Recursion from a Grammar

PROGRAM

```
#include &lt;stdio.h&gt;

int main() {
    char A;
    char alpha[20], beta[20];

    printf("Enter non-terminal: ");
    scanf(" %c", &amp;A);

    printf("Enter alpha (left recursive part): ");
    scanf("%19s", alpha);

    printf("Enter beta (non-left recursive part): ");
    scanf("%19s", beta);

    printf("%c -&gt; %s%c'\\n", A, beta, A);
    printf("%c' -&gt; %s%c' | epsilon\\n", A, alpha, A);

    return 0;
}
```

ALGORITHM

1. Read the non-terminal, recursive part alpha and non-recursive part beta.
2. Rewrite the original production as A -> beta A'.
3. Create the new production A' -> alpha A' | epsilon.
4. Display the transformed grammar.
5. Stop.

SAMPLE OUTPUT

```
Input: A -> Aα | β (A, α, β supplied)\nOutput: A -> βA'\nA' -> αA' | epsilon
```

19. Generate an Abstract Syntax Tree (AST)

PROGRAM

```
#include &lt;stdio.h&gt;
 #include &lt;stdlib.h&gt;

struct node {
    char data;
    struct node *left, *right;
};

struct node* create(char data) {
    struct node *n = malloc(sizeof(struct node));
    n-&gt;data = data;
    n-&gt;left = n-&gt;right = NULL;
    return n;
}

int main() {
    struct node *root = create('+');

    root-&gt;left = create('a');
    root-&gt;right = create('*');
    root-&gt;right-&gt;left = create('b');
```

```
    root-&gt;right-&gt;right = create('c');

    printf("AST for a+b*c:\\n");
    printf("       +\\n");
    printf("      / \\\\\\n");
    printf("    a     *\\n");
    printf("         / \\\\\\n");
    printf("       b    c\\n");

    return 0;
}
```

ALGORITHM

1. Create a root node for the + operator.
2. Create the left child for operand a.
3. Create the right child for the * operator.
4. Attach b and c as the children of *.
5. Display the AST structure.
6. Stop.

SAMPLE OUTPUT

```
Expression: a+b*c\nOutput: Root +; left child a; right child *; children b and c
```

20. Top-Down Parser Without Backtracking

PROGRAM

```
#include &lt;stdio.h&gt;

 char input[100];
 int i = 0;

 int E();
 int T();

 int F() {
     if (input[i] == 'i') {
         i++;
         return 1;
     }
     if (input[i] == '(') {
         i++;
         if (E() &amp;&amp; input[i] == ')') {
              i++;
              return 1;
         }
     }
     return 0;
 }

 int T1() {
     if (input[i] == '*') {
         i++;
         return F() &amp;&amp; T1();
     }
     return 1;
 }

 int T() {
     return F() &amp;&amp; T1();
 }

 int E1() {
     if (input[i] == '+') {
         i++;
         return T() &amp;&amp; E1();
     }
     return 1;
 }

 int E() {
     return T() &amp;&amp; E1();
 }

 int main() {
     printf("Enter expression (i+i*i): ");
     scanf("%99s", input);

     if (E() &amp;&amp; input[i] == '\\0')
          printf("Accepted");
     else
          printf("Rejected");

     return 0;
 }
```

ALGORITHM

1. Initialize the input position to zero.
2. Parse a factor: an identifier i or a parenthesized expression.
3. Parse a term containing factors separated by *.
4. Parse an expression containing terms separated by +.
5. Accept only if the entire input is consumed.
6. Otherwise reject the expression.

SAMPLE OUTPUT

```
Input: i+i*i\nOutput: Accepted
```